Lab 3 Complete Report

Smart Resource Allocation and Scheduling System using Greedy Algorithms

Name	Harinand S
Roll No.	2401201056
Specialisation	BCA (AI & Data Science)
Section	_____

Lab Name: Lab 3 — Smart Resource Allocation and Scheduling System using Greedy Algorithms

Question / Aim

Develop a Smart Resource Allocation and Scheduling System using greedy algorithms to solve resource allocation, activity scheduling, and data compression problems. Implement Fractional Knapsack, Activity Selection, and Huffman Coding; analyze execution time and memory use; and generate visualizations.

1. Introduction

A greedy algorithm constructs a solution by repeatedly selecting the locally best available choice. This lab demonstrates three classic applications: allocating divisible resources, scheduling compatible activities, and constructing variable-length prefix codes for compression.

2. Tools and Environment

Language: Python 3. Libraries: heapq and collections for Huffman Coding; time and tracemalloc for performance measurement; matplotlib for visualizations. Timing and memory numbers are measured on the execution environment used to generate this report and may differ on another computer.

3. Solution 1 — Fractional Knapsack

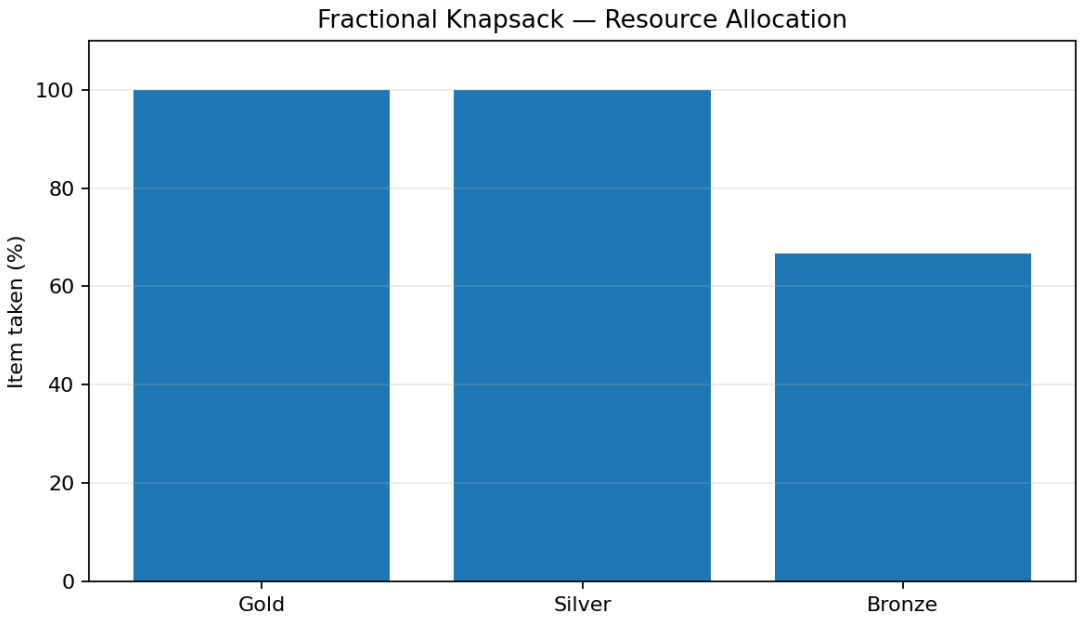
Problem: Given items with weights and values and a capacity, maximize value when fractions of items are allowed.

Greedy rule: Sort by value/weight ratio in descending order, take the highest-ratio items first, and take a fraction of the next item if capacity is insufficient.

Sample input: Gold (weight 10, value 60), Silver (20, 100), Bronze (30, 120), capacity 50. Ratios are 6, 5, and 4. The algorithm takes Gold and Silver fully and 2/3 of Bronze. Maximum value = 60 + 100 + 80 = 240.

Why greedy works: The value per unit weight is constant within each divisible item. If a solution uses weight from a lower-ratio item while a higher-ratio item remains available, exchanging that weight improves or preserves value. Repeating the exchange yields the ratio-sorted solution.

Complexity: O(n log n) time for sorting and O(n) auxiliary space for the sorted list and selection.



4. Solution 2 — Activity Selection

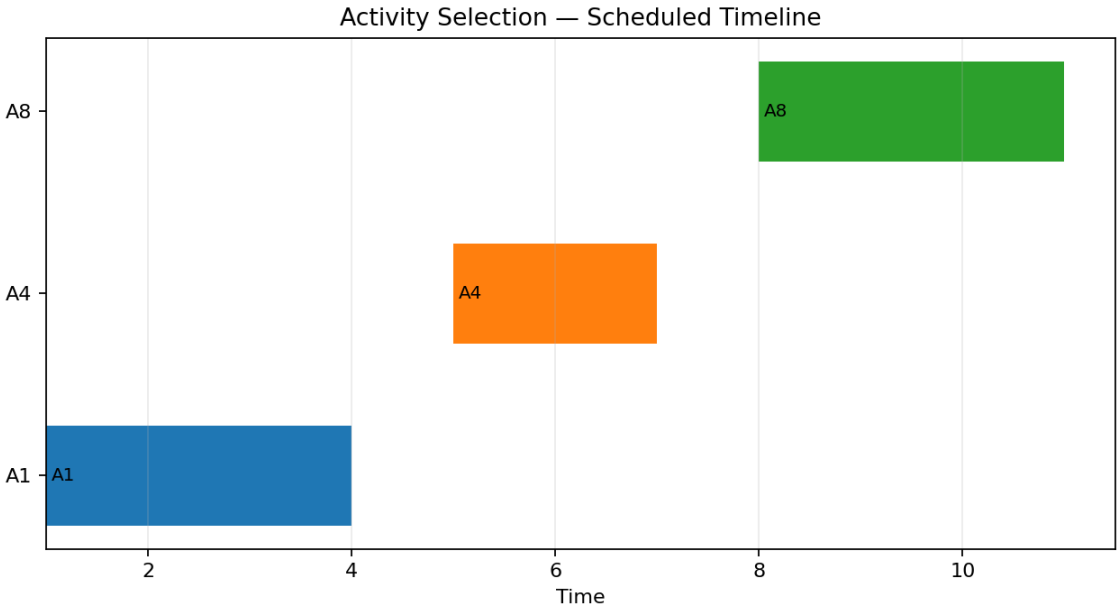
Problem: Select the maximum number of mutually compatible activities, each with a start and finish time, for one resource.

Greedy rule: Sort activities by finish time. Select the first activity, then select each activity whose start time is greater than or equal to the finish time of the last selected activity.

For the supplied sample, the selected schedule is A1 [1,4], A4 [5,7], A8 [8,11]. This gives three compatible activities.

Why greedy works: An activity that finishes earliest leaves at least as much time for all remaining activities as any activity that finishes later. Replacing the first activity of an optimal schedule with the earliest-finishing compatible activity cannot reduce the number of activities; the same argument applies to the remaining suffix.

Complexity: O(n log n) time for sorting, followed by an O(n) scan; O(n) auxiliary space for sorting and the selected list.



5. Solution 3 — Huffman Coding

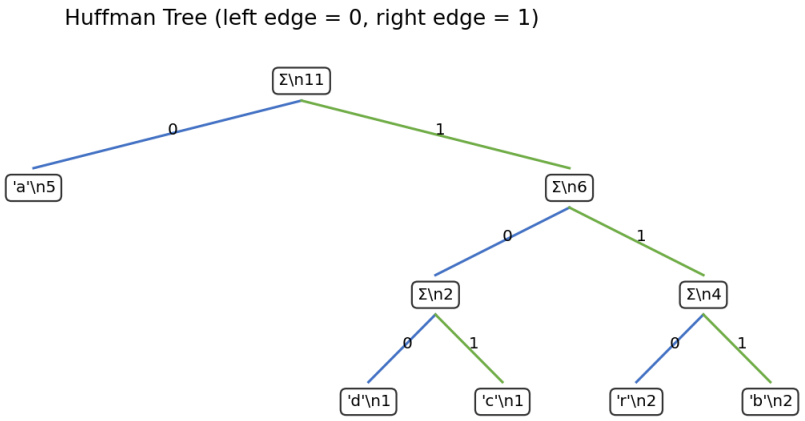
Problem: Create a prefix-free binary code from symbol frequencies so that frequent symbols generally use shorter codes and the total weighted encoded length is minimized.

Greedy rule: Put each symbol in a min-heap by frequency. Repeatedly remove the two least-frequent nodes, combine them as children of a new node with summed frequency, and reinsert the parent. Traverse left as 0 and right as 1 to generate codes.

Sample text: abracadabra. The generated code map is {'a': '0', 'b': '111', 'c': '101', 'd': '100', 'r': '110'} for this run. Encoded length = 23 bits versus 88 bits when each of the 11 characters is represented as 8 bits. The encoded/original ratio is 26.14%, excluding codebook/header storage.

Why greedy works: The two least-frequent symbols can be placed as deepest siblings in an optimal prefix tree without increasing the weighted path length. Merging them into one combined symbol reduces the problem to a smaller instance; recursively solving that instance gives an optimal Huffman tree.

Complexity: O(k log k) time and O(k) space, where k is the number of distinct symbols. Building codes traverses the tree.

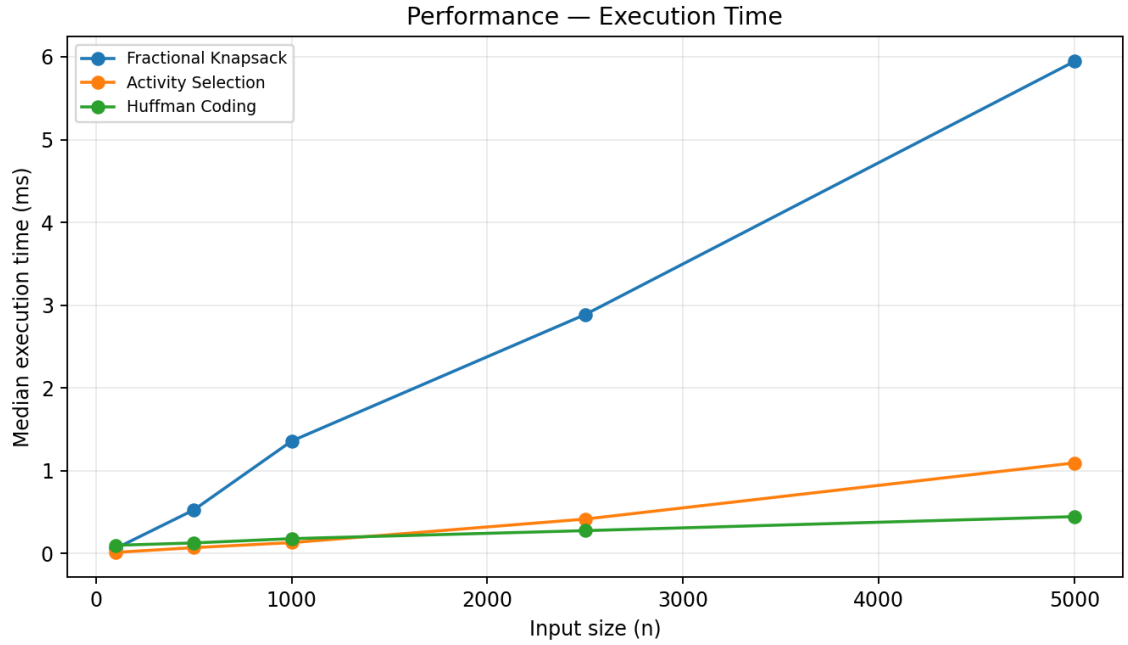


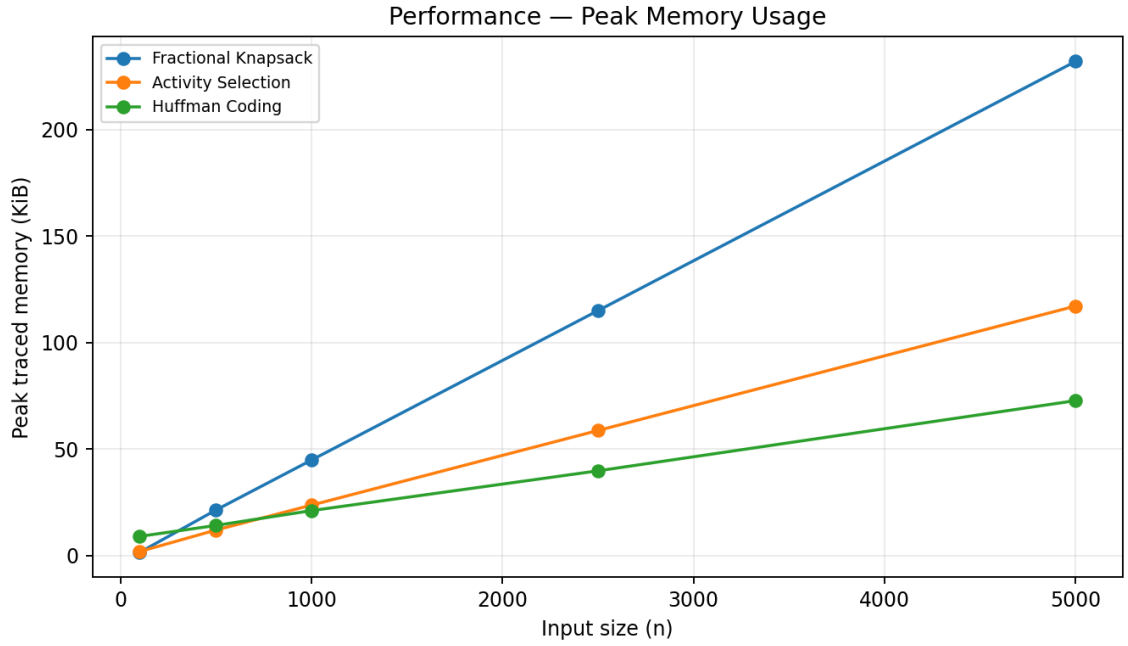
6. Measured Execution Time and Memory

For each input size, each algorithm was measured five times. The reported execution time and peak traced allocation are medians. The memory figure is tracemalloc's Python allocation peak, not total operating-system process memory.

Algorithm	n	Time (ms)	Peak memory (KiB)
Fractional Knapsack	100	0.0596	1.41
Activity Selection	100	0.0128	1.79
Huffman Coding	100	0.0983	8.92
Fractional Knapsack	500	0.5260	21.32
Activity Selection	500	0.0703	11.91
Huffman Coding	500	0.1269	14.07
Fractional Knapsack	1000	1.3580	44.77
Activity Selection	1000	0.1309	23.60
Huffman Coding	1000	0.1788	21.03
Fractional Knapsack	2500	2.8864	114.98

Activity Selection	2500	0.4153	58.66
Huffman Coding	2500	0.2755	39.72
Fractional Knapsack	5000	5.9446	232.02
Activity Selection	5000	1.0939	117.10
Huffman Coding	5000	0.4448	72.68





7. Complexity Comparison

Algorithm	Time complexity	Space complexity
Fractional Knapsack	O(n log n)	O(n)
Activity Selection	O(n log n)	O(n)
Huffman Coding	O(k log k)	O(k)

8. Practical Applications

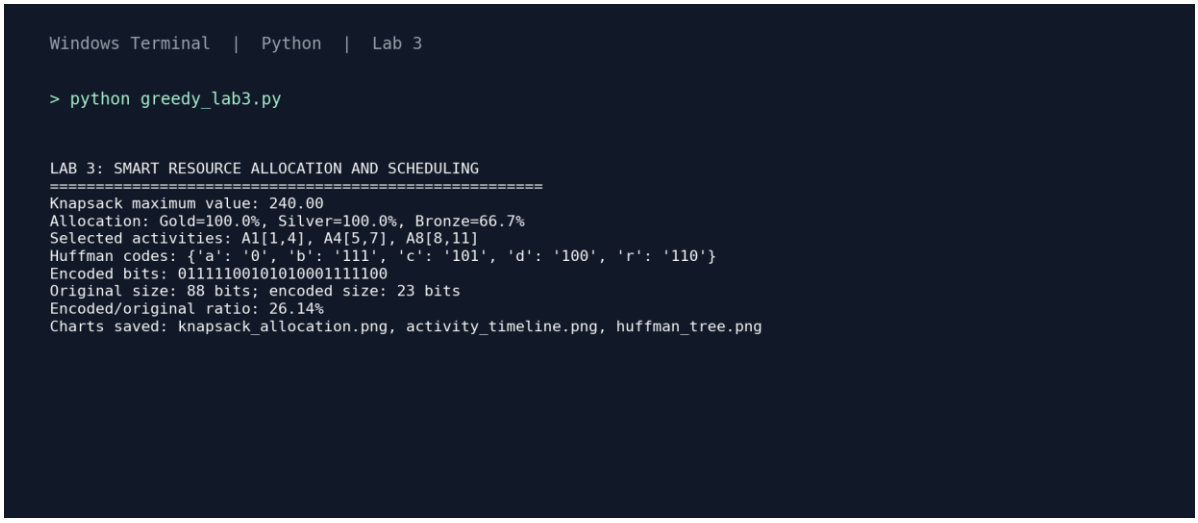
- Fractional Knapsack: allocating divisible budget, bandwidth, or material quantities when value per unit is known.
- Activity Selection: meeting-room booking, classroom/lab scheduling, and selecting non-overlapping tasks.
- Huffman Coding: lossless data compression and entropy coding used in formats and compression pipelines.

9. Observations and Limitations

- Greedy solutions are efficient when the problem has the appropriate greedy-choice property and optimal substructure.
- Fractional Knapsack permits fractions; the same ratio-based greedy strategy is not guaranteed optimal for 0/1 Knapsack, where items cannot be split.
- Activity Selection assumes one shared resource and that activities are compatible when the next start is at least the previous finish.
- Huffman coding's measured cost depends on the number of distinct symbols as well as input length. The sample compression ratio excludes the codebook and file-format overhead.
- Benchmark timings are affected by processor load, Python version, and interpreter overhead; use repeated runs and medians for fairer comparison.

10. Program Output Screenshot

The screenshot below was generated from the actual execution of the submitted Python program.



11. Conclusion

The lab implements three greedy algorithms and demonstrates their solutions, visualizations, empirical performance, and asymptotic complexities. Greedy methods can be fast and optimal for the specific structures studied, but their assumptions must be checked before applying them to a new optimization problem.

12. Source Code / Solution

Complete runnable source is provided in the attached project bundle at src/greedy_lab3.py; benchmark runner: src/benchmark_lab3.py. The source listing begins below.

```
"""
Lab 3: Smart Resource Allocation and Scheduling System using Greedy Algorithms
Student: Kushagra Verma | Roll No.: 2401201044
Specialisation: BCA (AI & Data Science) | Section: _________
Run: python greedy_lab3.py
```

```
"""
from collections import Counter
from dataclasses import dataclass
import heapq
import time
import tracemalloc
import random
import statistics
import matplotlib.pyplot as plt


def fractional_knapsack(items, capacity):
    """Return (maximum value, [(name, fraction_taken), ...])."""
    if capacity < 0:
        raise ValueError("Capacity must be non-negative.")
    if any(x["weight"] <= 0 for x in items):
        raise ValueError("Every item weight must be positive.")
    ordered = sorted(items, key=lambda x: x["value"] / x["weight"], reverse=True)
    total, chosen, remaining = 0.0, [], capacity
    for item in ordered:
        if remaining <= 0:
            break
        take = min(1.0, remaining / item["weight"])
        total += item["value"] * take
        remaining -= item["weight"] * take
        if take > 0:
            chosen.append((item["name"], take))
    return total, chosen


def activity_selection(activities):
    """Return maximum-size compatible activity set, sorted by finish time."""
    if not activities:
        return []
    ordered = sorted(activities, key=lambda a: a["finish"])
    chosen = [ordered[0]]
    last_finish = ordered[0]["finish"]
    for act in ordered[1:]:
        if act["start"] >= last_finish:
            chosen.append(act)
            last_finish = act["finish"]
    return chosen


@dataclass
class HuffmanNode:
    char: str | None
    freq: int
    left: object = None
    right: object = None
    def __lt__(self, other):
        return self.freq < other.freq


def build_huffman_tree(text):
    if not text:
        return None
    freq = Counter(text)
    heap = [HuffmanNode(ch, count) for ch, count in freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        heapq.heappush(heap, HuffmanNode(None, left.freq + right.freq, left, right))
    return heap[0]


def generate_codes(node, prefix="", codes=None):
    if codes is None:
        codes = {}
    if node is None:
        return codes
    if node.char is not None:
        codes[node.char] = prefix or "0"
    else:
        generate_codes(node.left, prefix + "0", codes)
        generate_codes(node.right, prefix + "1", codes)
    return codes


def huffman_encode(text):
    root = build_huffman_tree(text)
    codes = generate_codes(root)
    encoded = "".join(codes[ch] for ch in text)
    return encoded, codes, root


def plot_knapsack(selection, output="knapsack_allocation.png"):
    names = [name for name, _ in selection]
    vals = [fraction * 100 for _, fraction in selection]
    plt.figure(figsize=(7.5, 4.4))
    plt.bar(names, vals)
    plt.ylabel("Item taken (%)")
    plt.ylim(0, 110)
    plt.title("Fractional Knapsack — Resource Allocation")
    plt.grid(axis="y", alpha=.25)
    plt.tight_layout(); plt.savefig(output, dpi=160); plt.close()


def plot_activity_timeline(selected, output="activity_timeline.png"):
    plt.figure(figsize=(8, 4.5))
    for i, a in enumerate(selected):
```

```
        plt.barh(i, a["finish"]-a["start"], left=a["start"], height=.55)
        plt.text(a["start"]+.05, i, a["name"], va="center", ha="left", fontsize=9)
    plt.yticks(range(len(selected)), [a["name"] for a in selected])
    plt.xlabel("Time")
    plt.title("Activity Selection — Scheduled Timeline")
    plt.grid(axis="x", alpha=.25)
    plt.tight_layout(); plt.savefig(output, dpi=160); plt.close()


def plot_huffman_tree(node, output="huffman_tree.png"):
    fig, ax = plt.subplots(figsize=(9, 5.4))
    ax.axis("off")
    def layout(n, depth=0, x0=0, x1=1):
        if n is None: return
        x=(x0+x1)/2; y=1-depth*.19
        label = f"{n.char!r}\\n{n.freq}" if n.char is not None else f"Σ\\n{n.freq}"
        ax.text(x,y,label,ha="center",va="center",fontsize=9,
                bbox=dict(boxstyle="round,pad=.35",fc="white",ec="#333333"))
        if n.left:
            lx,ly=layout(n.left,depth+1,x0,x)
            ax.plot([x,lx],[y-.035,ly+.035],color="#4472C4",lw=1.4)
            ax.text((x+lx)/2,(y+ly)/2,"0",fontsize=9)
        if n.right:
            rx,ry=layout(n.right,depth+1,x,x1)
            ax.plot([x,rx],[y-.035,ry+.035],color="#70AD47",lw=1.4)
            ax.text((x+rx)/2,(y+ry)/2,"1",fontsize=9)
        return x,y
    layout(node)
    ax.set_xlim(-.02,1.02); ax.set_ylim(-.03,1.08)
    ax.set_title("Huffman Tree (left edge = 0, right edge = 1)")
    plt.tight_layout(); plt.savefig(output,dpi=160); plt.close()


def benchmark_once(fn):
    tracemalloc.start()
    start = time.perf_counter()
    fn()
    elapsed = time.perf_counter() - start
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return elapsed * 1000, peak / 1024


def benchmark():
    """Median of 5 measured runs; timings vary by machine and Python version."""
    rows=[]
    for n in [100, 500, 1000, 2500, 5000]:
        items=[{"name":f"I{i}","weight":random.randint(1,100)+.1,
                "value":random.randint(1,300)} for i in range(n)]
        acts=[]
        t=0
        for i in range(n):
            start=random.randint(0, n*2)
            acts.append({"name":f"A{i}","start":start,"finish":start+random.randint(1,20)})
        text="".join(random.choice("abcdefghijklmnopqrstuvwxyz ") for _ in range(n))
        cases=[
            ("Fractional Knapsack", lambda items=items: fractional_knapsack(items, n*20)),
            ("Activity Selection", lambda acts=acts: activity_selection(acts)),
            ("Huffman Coding", lambda text=text: huffman_encode(text)),
        ]
        for name, fn in cases:
            results=[benchmark_once(fn) for _ in range(5)]
            rows.append({"Algorithm":name,"Input size":n,
                         "Time (ms)":statistics.median(x[0] for x in results),
                         "Peak memory (KiB)":statistics.median(x[1] for x in results)})
    return rows


def main():
    items=[{"name":"Gold","weight":10,"value":60},
           {"name":"Silver","weight":20,"value":100},
           {"name":"Bronze","weight":30,"value":120}]
    value, selection=fractional_knapsack(items,50)
    activities=[{"name":"A1","start":1,"finish":4},{"name":"A2","start":3,"finish":5},
                {"name":"A3","start":0,"finish":6},{"name":"A4","start":5,"finish":7},
                {"name":"A5","start":3,"finish":9},{"name":"A6","start":5,"finish":9},
                {"name":"A7","start":6,"finish":10},{"name":"A8","start":8,"finish":11}]
    selected=activity_selection(activities)
    encoded,codes,root=huffman_encode("abracadabra")
    print("LAB 3: SMART RESOURCE ALLOCATION AND SCHEDULING")
    print("="*54)
    print(f"Knapsack maximum value: {value:.2f}")
    print("Allocation:", ", ".join(f"{n}={f*100:.1f}%" for n,f in selection))
    print("Selected activities:", ", ".join(f"{a['name']}[{a['start']},{a['finish']}]" for a in selected))
    print("Huffman codes:", dict(sorted(codes.items())))
    print("Encoded bits:", encoded)
    print(f"Original size: {len('abracadabra')*8} bits; encoded size: {len(encoded)} bits")
    print(f"Encoded/original ratio: {len(encoded)/(len('abracadabra')*8):.2%}")
    plot_knapsack(selection, "knapsack_allocation.png")
    plot_activity_timeline(selected, "activity_timeline.png")
    plot_huffman_tree(root, "huffman_tree.png")
    print("Charts saved: knapsack_allocation.png, activity_timeline.png, huffman_tree.png")

if __name__ == "__main__":
    main()
```

13. GitHub Repository

https://github.com/Harinand07/Algorithm-Performance-Measurement-and-Benchmarking-Tool/tree/main/Lab2_Algorithm_Performance_Tool